Module 3

Acquiring External Data in Python

Data Analysis

Burkina Institute of Technology - L3 Computer Science
B. Tanguy KABORE

What we will see

- The three structured formats you will meet: CSV, JSON, XML
- Reading and writing files with Python and with pandas
- Turning each format into a DataFrame
- Downloading data from a URL or a public API

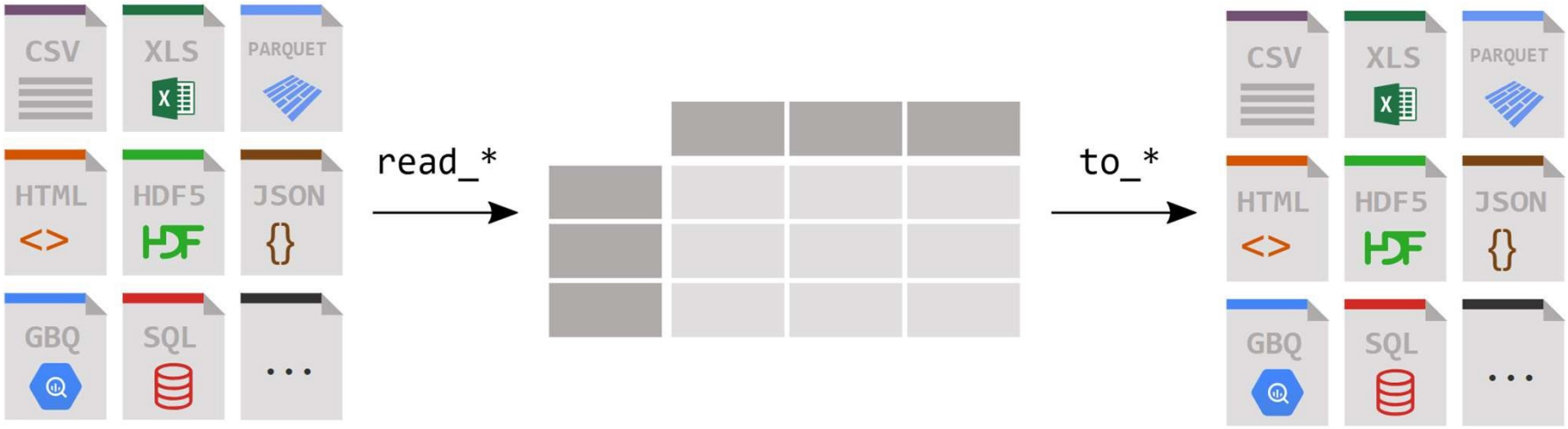
Duration: 5 hours. Until now we typed our data by hand. Real data comes from files and from the network.

Where does data come from?

- Exports from software: Excel, accounting tools, school management systems
- Open data portals: government statistics, weather, health
- Web APIs: a URL that returns JSON when you ask for it
- Sensors and logs: one line per event, often CSV

In every case, the data arrives as text in a known structure. Our job is to parse that structure.

Reading and writing data



Source: pandas documentation, "How do I read and write tabular data?" - BSD-3-Clause licence.

CSV

Simple definition

CSV (Comma-Separated Values) is a plain text file: one line per row, values separated by a delimiter, usually a comma.

```
name,age,city
Awa,21,Ouagadougou
Issa,23,Bobo-Dioulasso
```

Comparison

A CSV is a spreadsheet with all the formatting removed. Only the cells remain.

Reading a CSV with pandas

```
import pandas as pd
df = pd.read_csv("students.csv")
df.head()
```

```
   name  age            city
0   Awa   21     Ouagadougou
1  Issa   23  Bobo-Dioulasso
```

The first line becomes the column names. Everything else becomes rows. pandas guesses the types: `age` is `int64`, the two others are `object`.

Error: file not found

```
pd.read_csv("students.csv")
```

```
FileNotFoundError: [Errno 2] No such file or directory: 'students.csv'
```

Cause: Python looks in the **current working directory**, which is not always the folder of your notebook.

Fix: run `import os; os.getcwd()` to see where you are, then use a relative path like `"data/students.csv"` or move the file.

Error: everything in one column

French Excel exports use `;` as a separator. pandas does not guess:

```
            name;age;city
0      Awa;21;Ouagadougou
1  Issa;23;Bobo-Dioulasso
```

Fix:

```
pd.read_csv("students.csv", sep=";")
```

Also check `decimal=","` if numbers look like `15,5`, otherwise they are read as text.

Error: encoding

```
UnicodeDecodeError: 'utf-8' codec can't decode byte 0xe9 in position 3:
unexpected end of data
```

Cause: the file was saved in Windows encoding (cp1252 or latin-1) and contains accented characters like `é`. pandas expects UTF-8 by default.

Fix:

```
pd.read_csv("students.csv", sep=";", encoding="latin-1")
```

When you create files yourself, always save them as UTF-8.

Error: a line with too many fields

```
ParserError: Error tokenizing data. C error:
Expected 3 fields in line 3, saw 5
```

Cause: line 3 contains extra separators, often a comma inside a text value that was not quoted, like `Ouagadougou, Secteur 15`.

Fix: open the file in a text editor and look at that line. Then either fix the file, or read with `on_bad_lines="warn"` to skip the bad rows and see which ones.

The read_csv options you will really use

Option	Use
`sep=";"`	Separator other than comma
`encoding="latin-1"`	Accented characters from Windows
`decimal=","`	Numbers written `15,5`
`parse_dates=["date"]`	Convert a column to datetime at load time
`na_values=["N/A", "-"]`	Extra strings to treat as missing

There are many more; `help(pd.read_csv)` lists them. These five solve most problems.

Writing a CSV

```
df.to_csv("results/clean_students.csv", index=False, encoding="utf-8")
```

`index=False` avoids writing the 0, 1, 2 index as a useless first column. Open the file in a text editor once to check that it looks like you expected.

Writing Excel works the same way: `df.to_excel("results.xlsx", index=False)`. It needs the `openpyxl` package installed.

Reading CSV without pandas

Sometimes you need line by line control, for example for a huge file:

```
import csv
with open("students.csv", newline="", encoding="utf-8") as f:
    reader = csv.DictReader(f)
    for row in reader:
        print(row["name"], row["age"])
```

Each `row` is a dictionary. The `csv` module is part of the standard library. The `with` block closes the file for you, even if an error occurs.

JSON

Simple definition

JSON (JavaScript Object Notation) is a text format made of objects `{}` with key-value pairs and lists `[]`. It maps directly to Python dictionaries and lists.

```
[{"name": "Awa", "age": 21},
 {"name": "Issa", "age": 23}]
```

Almost every web API speaks JSON. It can be nested, which CSV cannot.

JSON in Python: the json module

```
import json

with open("students.json", encoding="utf-8") as f:
    data = json.load(f)        # file -> Python objects

type(data)        # list
data[0]["name"]   # 'Awa'
```

`json.load` reads from a file, `json.loads` reads from a string. `json.dump` and `json.dumps` do the reverse.

Writing JSON

```
record = {"school": "BIT",
          "students": [{"name": "Awa", "grade": 15.5}]}

with open("export.json", "w", encoding="utf-8") as f:
    json.dump(record, f, indent=2, ensure_ascii=False)
```

`indent=2` makes the file readable. `ensure_ascii=False` keeps accented characters as they are instead of `\u00e9`.

Error: invalid JSON

```
json.loads("{'name': 'Awa'}")
```

```
JSONDecodeError: Expecting property name enclosed in double quotes:
line 1 column 2 (char 1)
```

Cause: JSON requires double quotes. Single quotes are Python, not JSON.

Another classic: `JSONDecodeError: Expecting value: line 1 column 1 (char 0)` means the text is empty or is an HTML error page instead of JSON. Print the first 200 characters to check.

JSON to DataFrame

For a list of flat records, pandas does it in one line:

```
df = pd.read_json("students.json")
```

For nested JSON, flatten it first:

```
data = json.load(open("api_result.json", encoding="utf-8"))
df = pd.json_normalize(data["results"])
```

`json_normalize` turns nested keys into columns named `address.city`, `address.zip`, and so on.

XML

Simple definition

XML uses tags, like HTML, to describe a tree of elements with attributes and text.

```
<students>
  <student id="1"><name>Awa</name><age>21</age></student>
  <student id="2"><name>Issa</name><age>23</age></student>
</students>
```

Older than JSON, still common in administration, banking and RSS feeds.

Parsing XML with the standard library

```
import xml.etree.ElementTree as ET

tree = ET.parse("students.xml")
root = tree.getroot()
for st in root.findall("student"):
    print(st.get("id"), st.find("name").text, st.find("age").text)
```

```
1 Awa 21
2 Issa 23
```

`get()` reads an attribute, `find().text` reads the text of a child tag.

XML to DataFrame

For a regular XML like ours, pandas can do it directly:

```
df = pd.read_xml("students.xml")
```

```
   id  name  age
0   1   Awa   21
1   2  Issa   23
```

`read_xml` uses the `lxml` package by default; without it, add `parser="etree"`. For irregular or deeply nested XML, go through `ElementTree` and build a list of dictionaries yourself.

Which format when?

CSV

- Flat tables
- Opens in Excel
- Smallest files
- No types, no nesting

JSON / XML

- Nested or hierarchical data
- Web APIs (JSON), feeds (XML)
- Keeps structure and some types
- More verbose

When you have a choice, store clean tables as CSV and keep the raw API answer as JSON.

Exercise 1 - Files (30 minutes)

Do it now

1. Save the grades table from Module 1 as `grades.csv` with `;` as separator, then read it back correctly
2. Write the same table as `grades.json` with `df.to_json(orient="records", indent=2)` and open it in a text editor
3. Load the JSON with the `json` module and print the name of the second student
4. Build a small XML file by hand with 3 students and read it with `pd.read_xml`

Getting data from the web

Two ways:

1. A URL that points to a file: give it directly to pandas
2. An **API**: a URL with parameters that returns JSON built for you

```
url = "https://raw.githubusercontent.com/mwaskom/seaborn-data/master/tips.csv"
tips = pd.read_csv(url)
tips.shape        # (244, 7)
```

pandas downloads and parses in one step. Needs an internet connection, obviously.

What is an API?

Simple definition

A **web API** is a server that answers structured questions sent as URLs. You ask for "the forecast for these coordinates", it returns JSON.

Comparison

A website is a shop window designed for humans. An API is the back door designed for programs: no layout, just the goods.

The requests library

```
import requests

url = "https://api.open-meteo.com/v1/forecast"
params = {"latitude": 12.37, "longitude": -1.53,
          "daily": "temperature_2m_max",
          "timezone": "Africa/Abidjan", "forecast_days": 3}

r = requests.get(url, params=params, timeout=10)
r.status_code      # 200
data = r.json()
```

Open-Meteo is a free weather API without a key. Coordinates here are Ouagadougou. `requests` is not in the standard library: `pip install requests`.

From the answer to a DataFrame

```
data["daily"]
```

```
{'time': ['2026-10-22', '2026-10-23', '2026-10-24'],
 'temperature_2m_max': [38.1, 36.5, 37.4]}
```

```
forecast = pd.DataFrame(data["daily"])
forecast["time"] = pd.to_datetime(forecast["time"])
```

Values will differ on your day, of course. The structure stays the same, and that is what your code depends on.

Checking the status code

Code	Meaning	What to do
200	OK	Parse the answer
400	Bad request	Check your parameters
401 / 403	Not allowed	API key missing or invalid
404	Not found	Check the URL
429	Too many requests	Slow down, wait
500	Server error	Not your fault; retry later

`r.raise_for_status()` raises an exception for anything 400 or above. Call it right after `requests.get`.

Error: no connection

```
ConnectionError: HTTPSConnectionPool(host='nonexistent.invalid', port=443):
Max retries exceeded with url: /data.json
```

Cause: wrong host name, no internet, or a proxy on the campus network.

Fix: check the URL in a browser first. In the lab, if the network is down, use the local copy of the file I will give you. Your notebook should not depend on the network to run: save the raw answer once.

Error: HTTP error

```
r = requests.get("https://httpbin.org/status/404", timeout=10)
r.raise_for_status()
```

```
HTTPError: 404 Client Error: Not Found for url: https://httpbin.org/status/404
```

The request reached the server, but the server said no. Read the code, read the documentation of the API, fix the URL or parameters.

Good habits with APIs

1. Always set a `timeout`, otherwise a dead server freezes your notebook
2. Save the raw JSON to disk the first time; work from the file afterwards
3. Respect rate limits: add `time.sleep(1)` in loops
4. Never write an API key in a notebook you share; read it from an environment variable

```
import os
key = os.environ.get("MY_API_KEY")
```

Downloading a file

```
r = requests.get(url, timeout=30)
r.raise_for_status()
with open("data/raw.csv", "wb") as f:
    f.write(r.content)
```

`"wb"` means write bytes: we do not decode, we save exactly what arrived. Use `urllib.request.urlretrieve(url, path)` from the standard library if you cannot install `requests`.

The acquisition pipeline

```
 source          raw copy            clean table        analysis
(API, file)  ->  data/raw/*.json  ->  data/clean/*.csv  ->  notebook
```

Keep this separation in your project. Raw files are never modified. Clean files can be regenerated from raw files by a script. If a step breaks, you know where to look.

Practical work 3 - From the web to a table (1h30)

Goal

Build a small dataset from an online source, save it, clean it and produce a first chart.

Material

Open-Meteo API (no key), or a CSV from an open data portal of your choice validated by me. Internet access in the lab, with a local fallback file.

Work in pairs. Deliverable: notebook plus `data/raw` and `data/clean` folders.

Practical work 3 - Steps

1. Query Open-Meteo for 7 days of daily max and min temperatures for two cities (Ouagadougou and Bobo-Dioulasso)
2. Save each raw answer as JSON in `data/raw/`
3. Build one DataFrame with columns `city`, `date`, `t_max`, `t_min` from the saved files
4. Export it to `data/clean/weather.csv`

Practical work 3 - Analysis

1. Reload `weather.csv` with `parse_dates=["date"]`
2. Compute the average max temperature per city
3. Draw a line plot with one line per city, labelled, with units
4. Write 3 lines: which city is hotter this week, and by how much?

Practical work 3 - Checks before you submit

- The notebook runs without network if the raw JSON files are present
- `read_csv` of the clean file gives the right dtypes (dates as datetime)
- No API key or personal path in the notebook
- The chart has a title, labels, units and a legend

Summary

What you know now

- `read_csv` with `sep`, `encoding`, `decimal`, `parse_dates` solves most file problems
- `json` module and `read_json` / `json_normalize` for JSON; `ElementTree` and `read_xml` for XML
- `requests.get` with `timeout` and `raise_for_status` for APIs
- Keep raw and clean data separate

Next module

Module 4: Statistical Analysis in Python

References

- pandas, "IO tools": https://pandas.pydata.org/docs/user_guide/io.html
- Python documentation, `csv`, `json`, `xml.etree.ElementTree` modules: https://docs.python.org/3/library/
- Requests documentation: https://requests.readthedocs.io/
- Open-Meteo API documentation: https://open-meteo.com/en/docs

References - continued

- A. Boschetti, L. Massaron, *Python Data Science Essentials*, 3rd ed., Packt, 2018, chapter 2 "Data Munging"
- Examples tested with pandas 2.2.3 and requests 2.34; the Open-Meteo values shown were retrieved on 22 October 2026

Questions?

You receive a CSV where every value is in one column and the accents are broken. Which two options do you add to `read_csv`, in which order do you test them?